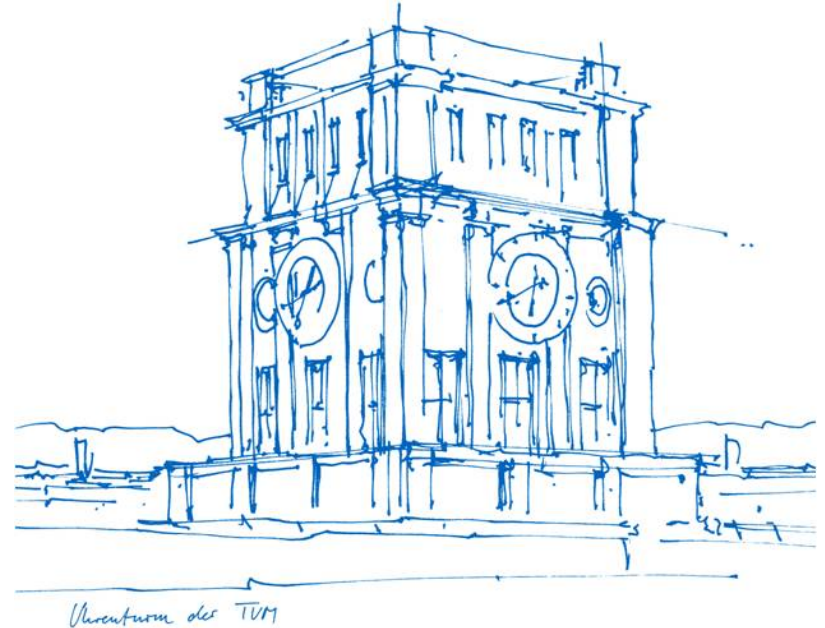


Memory Controller (Gruppe T028)

Efe Yörük, Arda Şahin, Fırat Yöndem

Technische Universität München

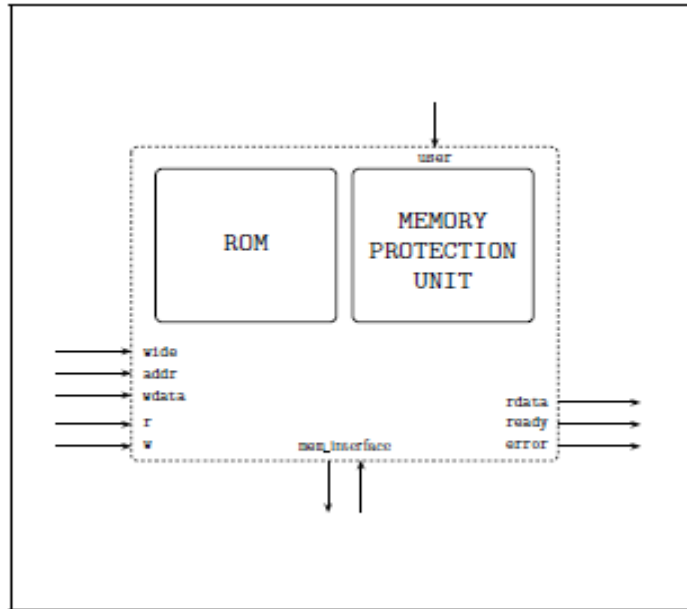
München, 17.08.2026



Was ist ein Memory Controller?

Definition:

Ein Memory Controller (MCC/MCU) ist eine digitale Schaltung, die den Datenfluss zwischen CPU und Hauptspeicher koordiniert und kontrolliert.



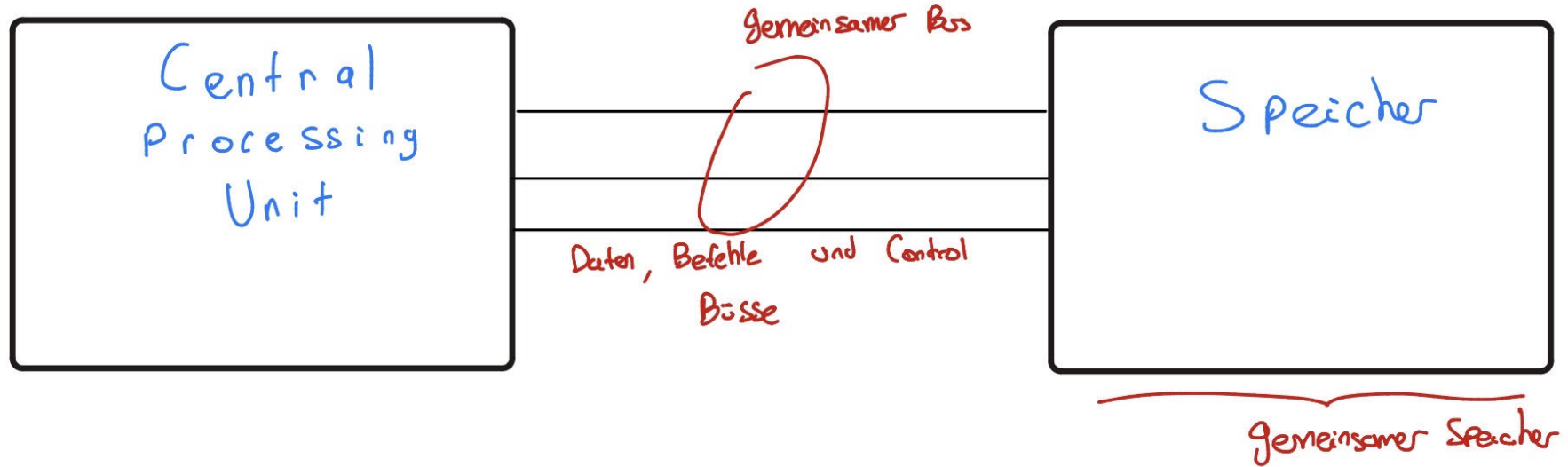
Warum brauchen wir einen Memory Controller?

- **Schutz :**
unkontrollierte Speicherzugriffe führen zu Datenverlust, Crashes und Sicherheitslücken
- **Koordination:**
verschiedene Speichertypen (ROM, RAM) brauchen einen gemeinsamen Vermittler
- **Abstraktion:**
die CPU muss nicht wissen wo Daten physisch liegen, der MC übernimmt das

Von Neumann Architektur

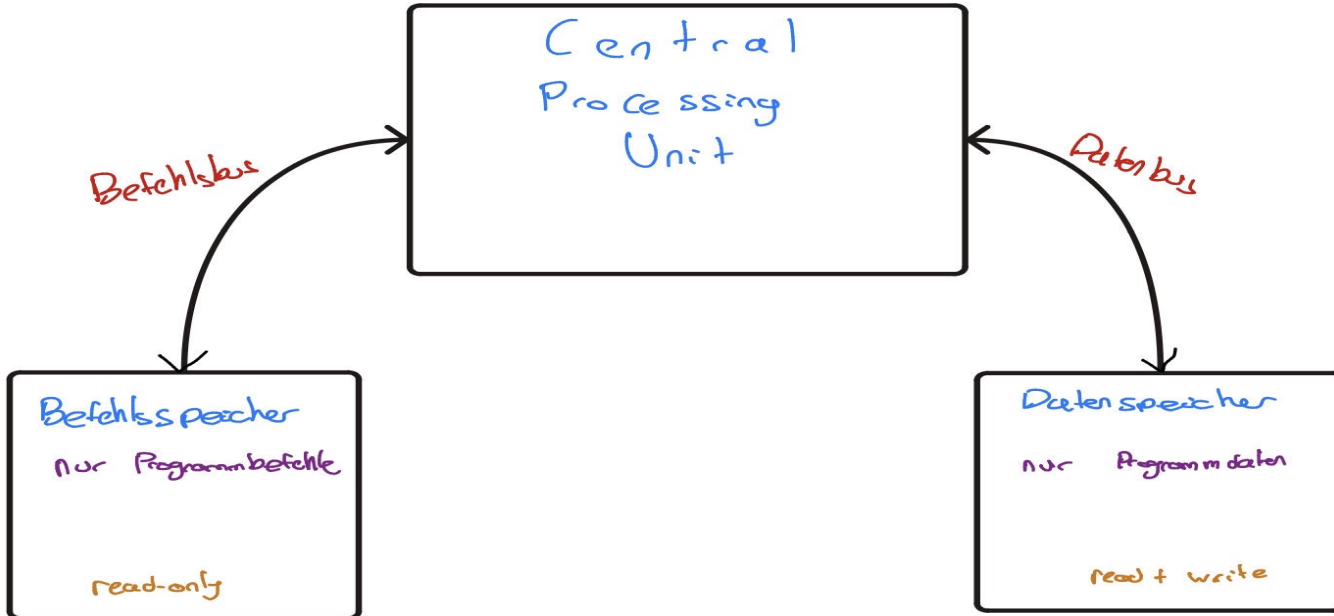
Flaschenhals dieser Architektur:

Daten und Befehle teilen sich einen Bus und einen Speicher



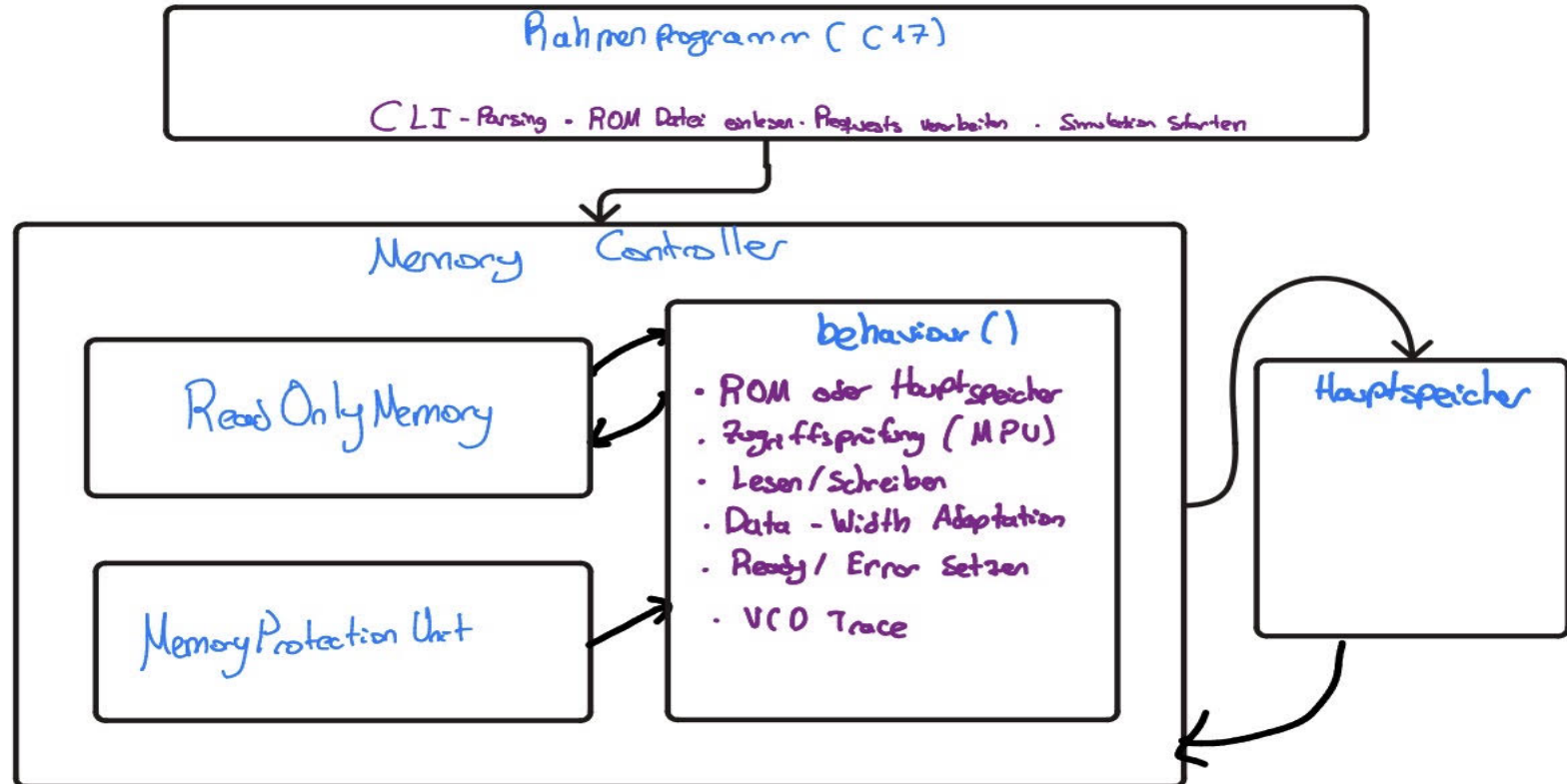
Harvard Architektur

- Zwei getrennte Busse → CPU kann gleichzeitig einen Befehl holen und auf Daten zugreifen
- Befehlsspeicher ist typischerweise read-only (wie ein ROM), Datenspeicher read + write



Unser Design

Modulstruktur

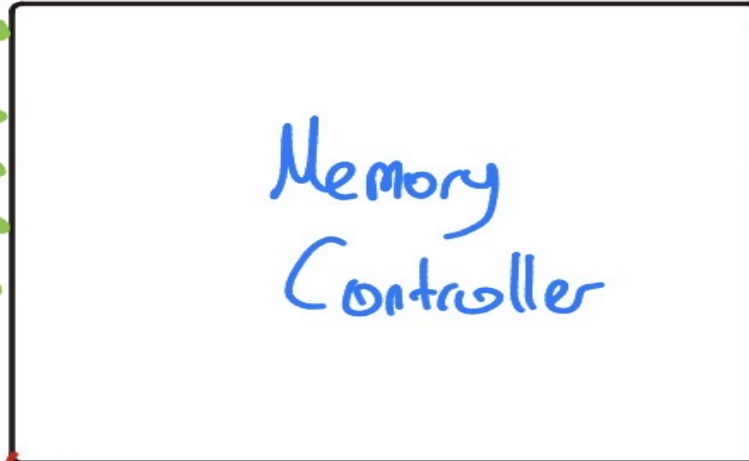
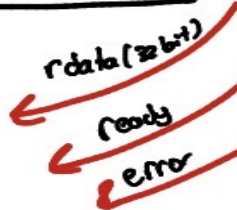


Eingaben/Ausgaben

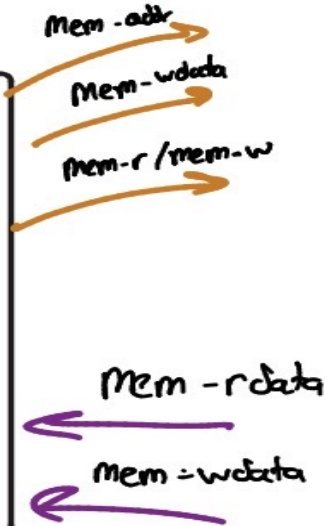
Von der CPU



An die CPU



Hauptspeicher



Eingabe
(CPU → MC)

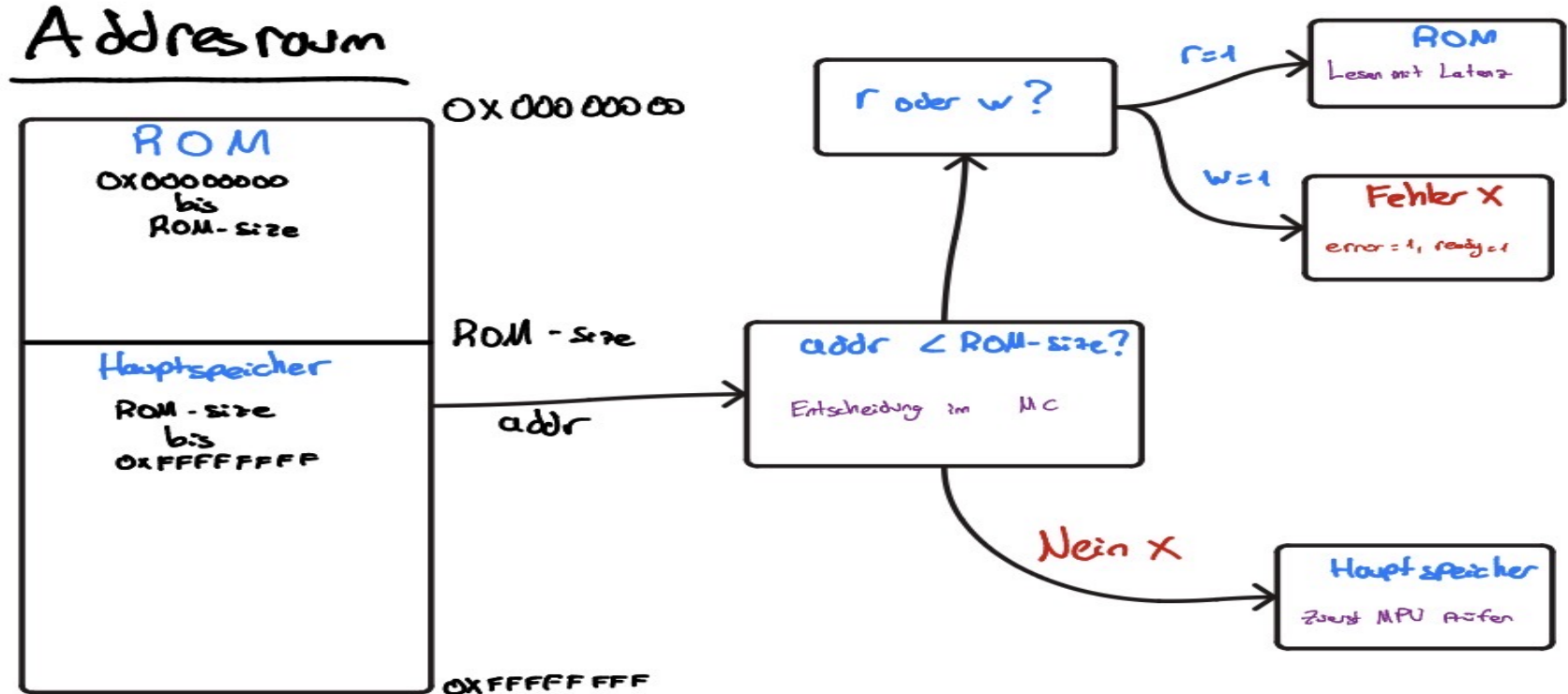
Ausgabe
(MC → CPU)

(MC → Speicher)

(Speicher → MC)

Funktionalität

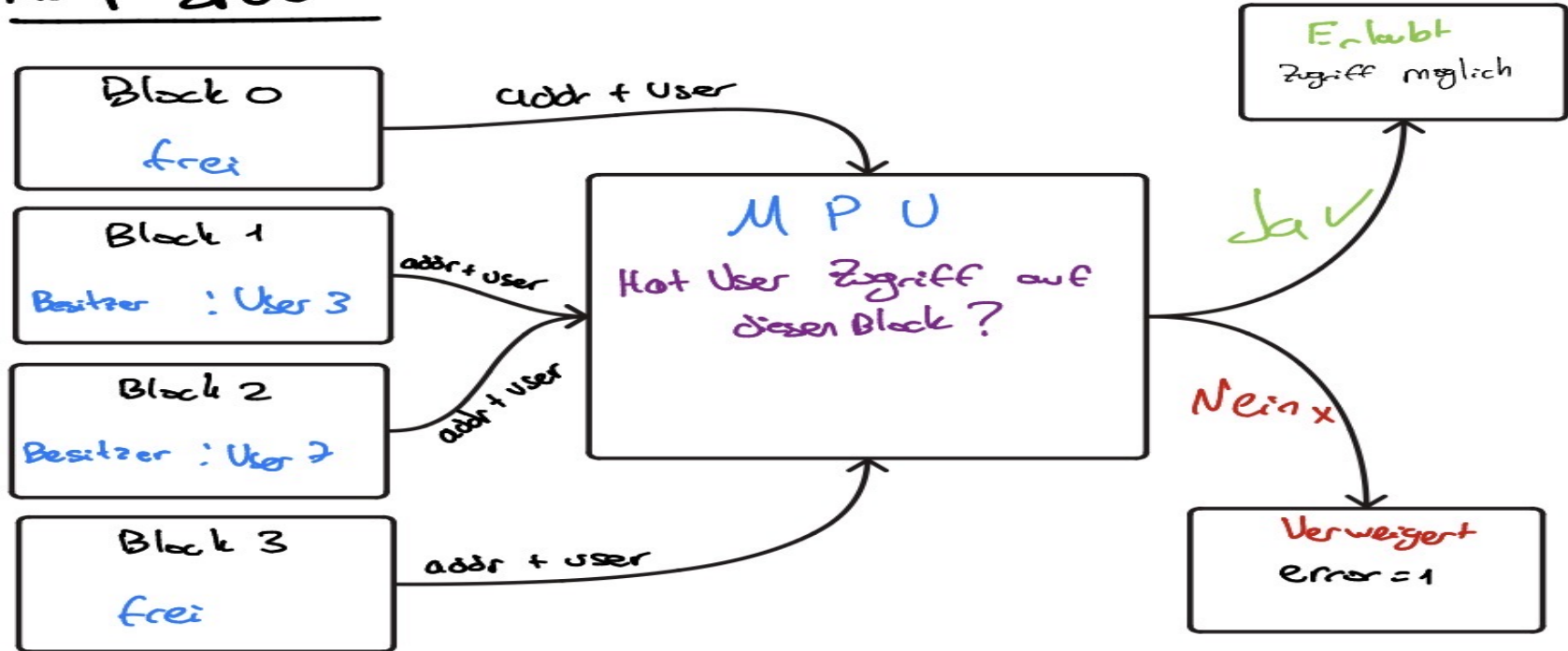
Memory Mapping



Funktionalität

Memory Protection Unit

Hauptspeicher



Funktionalität

Data-Width Adaptation

wide = T $\rightarrow$ 4 Byte Zugriff

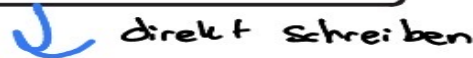
Lesen



rdata = B3 B2 B1 B0

Schreiben

rdata = B3 B2 B1 B0



wide = F $\rightarrow$ 1 Byte Zugriff

Lesen



rdata = 0x000000 B0

Schreiben

altes Wort lesen
B3 B2 B1 B0 (alt)



B0 ersetzen, zurückschreiben
B3 B2 B1 erhalten bleiben

Vergleich unserer Implementierung mit der Literaturrecherche

Gemeinsamkeiten mit echten Systemen:

- **Memory Mapping:** gemeinsamer Adressraum für verschiedene Speichertypen, genau wie in echten Von-Neumann-Systemen
- **MPU:** Zugriffskontrolle wie in echten CPUs (z.B. ARM Cortex-M)
- **Schreibschutz auf ROM:** standard in allen echten Systemen
- **Fehlerbehandlung:** error Signal wie echte Bus Fault Exceptions

Vergleich unserer Implementierung mit der Literaturrecherche

Unterschiede zu echten Systemen:

- Keine Signalqualität, Stromverbrauch, Hitze
- Unser MC folgt Von Neumann, moderne CPUs nutzen oft modifizierte Harvard Architektur (getrennte Caches)
- Keine Cache-Hierarchie
- Hauptspeicher hat feste 100-Zyklen Latenz — in Realität viel komplexer

Herausforderungen

- Timing-Probleme:

Neue Requests starteten bevor alte Signalwerte geschrieben wurden

→ gelöst durch wait()

- 1-Byte Schreiben:

Hauptspeicher unterstützt nur 4-Byte Zugriffe

→ erst lesen, Byte ersetzen, zurückschreiben (2 Speicherzugriffe)

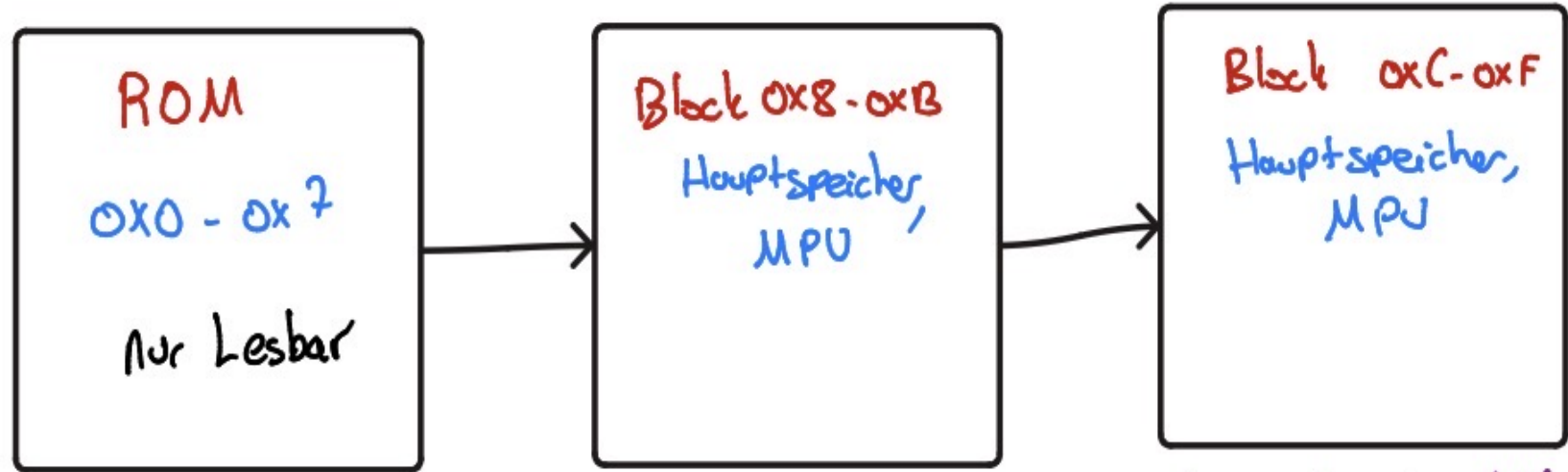
- ROM Latenz:

Warten bis rom_ready high wird, dann warten bis es wieder low wird

→ Synchronisation mit Submodul

Herausforderungen

- Submodul Synchronisation:
ROM und MPU müssen korrekt mit dem Hauptmodul kommunizieren
→ interne Signale `rom_read` und `allowed`
- Data-Width Adaptation:
wide=F bei Lesen
→ LSB aus 32-Bit Wort extrahieren, restliche Bytes auf 0 setzen



Jeder Speicherblock umfasst 4 Byte und wird durch die MPU verwaltet
Speicherzugriff auf ROM → sofort Fehler

MPU Test – Schreiben und Lesen

Anfragen 1-4 Speicher vorher

0x8: 0x00000000
frei

0xC: 0x00000000
frei

Anfragen

W 0x8 0xDEADBEEF Benutzer 1 (4 Byte) → ok
W 0xCAD Benutzer 1 (4 Byte) → ok
R 0x8 Benutzer 1 → 0xDEADBEEF → ok
W 0xC Benutzer 1 → 0x000000CAD → ok

↓ Speicher nachher

0x8: 0xDEADBEEF - Benutzer 1

0xC: 0x000000CAD - Benutzer 1

MPU Test – Besitzwechsel + 1-Byte

Anfragen 5-8 : MPU +1 Byte Schreiben

Speicher vorher

0x8: DEAD BEEF
Besitzer: Benutzer 1

0xC: 00000 CAD
Besitzer: Benutzer 1

Anfragen

R 0x8 Benutzer 2 - kein Besitzer → Zugriffsfehler

W 0x8 0x AA Benutzer 0 (1 Byte) - übernimmt Besitz → ok

W 0x8 0xDAD Benutzer 2 - Block gehört Benutzer 0 → Zugriffsfehler

W 0x8 0xDAD Benutzer 1 - Block gehört Benutzer 0 → Zugriffsfehler

↓ Speicher nachher

0x8: 0xDEADBEEF AA
Besitzer: Benutzer 0

0xC: 0x 00000 CAD
Besitzer: Benutzer 1

MPU Test — User 255 gibt frei

Anfragen 9-13: User 255 gibt Block frei

Speicher vorher

0x8: DEAD BEAF
Besitzer: Benutzer 0

0xC: 00000 CAD
Besitzer: Benutzer 1

Anfragen

W 0xC 0x11111111 Benutzer 255 - Block wird freigegeben → ok

R 0xC Benutzer 1 → 0x11111111 - freier Block, lesbar → ok

W 0xC 0xCAED Benutzer 2 - übernimmt den freien Block

R 0xC Benutzer 2 → 0x0000CAED → ok

R 0xC Benutzer 1 - Block gehört Benutzer 2 → Zugriffsfehler

↓ Speicher nachher

0x8: 0xDEADBEEF
Besitzer: Benutzer 0

0xC: 0x00000 CAD
Besitzer: Benutzer 2

ROM Test

ROM Test (ROM-size=0xC, Block-size=0x4)

ROM Inhalt

0x0
0x11221111

0x4
0x22222222

0x8
0x33333333

Anfragen

R	0x0	Benutzer 1	wide = T → rdata 0x11 22 1111	✓	Zyklus 6
R	0x4	Benutzer 1	wide = T → rdata 0x2222 2222	✓	Zyklus 13
R	0x8	Benutzer 1	wide = T → rdata 0x3333 3333	✓	Zyklus 20
W	0x0	0x12345678	Benutzer 1 wide = T → Fehler! ROM x		Zyklus 22
W	0x4	0xFF	Benutzer 1 wide = T → Fehler! ROM x		Zyklus 24
R	0x0	Benutzer 1	wide = T → rdata 0x11 22 1111	✓ (unverändert)	Zyklus 31
R	0x4	Benutzer 2	wide = F → rdata 0x000000 22	✓ (LSB)	Zyklus 38

Metrik - Taktzyklen

- **Erfolgreicher ROM-Lesezugriff:** ROM-Latenz + 6 Taktzyklen
- **Fehlerhafter ROM-Zugriff (Schreiben):** 2 Taktzyklen
- **Erfolgreicher Hauptspeicherzugriff:** Hauptspeicher-Latenz + 6 Taktzyklen (~*2 für 1-Byte Schreibzugriffe)
- **Fehlerhafter Hauptspeicherzugriff:** 2 Taktzyklen

Metrik – Vergleich mit realer Hardware

- **Fehlerfall (2 Zyklen):** reale MPUs arbeiten parallel → kaum Zusatzkosten. Bei uns → ein synchroner Funktionsaufruf statt eines eigenen Bus-Zugriffs (mit vergleichbar vernachlässigbarer Latenz).
- **6-Zyklen-Overhead:** ähnlich dem Protokoll-Overhead realer Bus-Standards (z.B. AXI benötigt für eine gepipelinte Transaktion ~3 Zyklen) nicht-gepipelinte Einzelzugriffe wie hier liegen oft höher.

- **Memory Controller** besteht aus einem Hauptmodul und zwei Submodulen:
ROM und Memory Protection Unit
- **Memory Mapping:**
Adressraum aufgeteilt in ROM (0x0 – rom_size) und Hauptspeicher
- **Memory Protection Unit:**
blockbasierte Zugriffskontrolle pro Benutzer
- **Data-Width Adaptation:**
4-Byte und 1-Byte Zugriffe werden unterstützt
- Fehlerhafte Zugriffe werden erkannt und mit error=1 signalisiert
- Rahmenprogramm in C17: CLI-Parsing, CSV-Eingabe, VCD-Trace

Danke für Ihre Aufmerksamkeit!